

CFD on HPC – OpenFOAM

03 – Advanced usage



Aleksander GRM



Introduction

Hydrofoil case

Simple mixer case

Ship resistance case

Propeller – Open water test case

Introduction



- ▶ **Templates:** use templates to start case
`$> cp $FOAM_ETC/templates/... case_dir`
- ▶ **Dictionaries:** use foamGet to copy templated dictionary files
`$> foamGet forcesIncompressible`
- ▶ **Information:** use foamInfo to get help on foam item
`$> foamInfo incompressibleFluid`
- ▶ **Dictionary manipulation:** use foamDictionary to edit dictionary files
`$> foamDictionary [OPTIONS] <dictionary file>`



Nice example in folder

- day_02/02_test_case-cavity_with_hole/snappyHexMesh/steadyState/
look into readme.txt file

Example

Fancy corrections of constant/polyMesh/boundary file symmetryPlane -> empty

- foamDictionary constant/polyMesh/boundary -keywords
- foamDictionary constant/polyMesh/boundary -entry entry0 -keywords

Front:

- foamDictionary constant/polyMesh/boundary -entry entry0/front -keywords
- foamDictionary constant/polyMesh/boundary -entry entry0/front/type -value
- foamDictionary constant/polyMesh/boundary -entry entry0/front/type -set empty

here are two choices, both works

- foamDictionary constant/polyMesh/boundary -entry entry0/front/inGroups -set "List<word> 1(empty)"
- foamDictionary constant/polyMesh/boundary -entry entry0/front/inGroups -remove

Hydrofoil case



- ▶ describe basic case philosophy
- ▶ describe two different meshing approaches
 - ▶ OpenFOAM foil **blockMesh**: change angle of attack with velocity vector rotation
 - ▶ GMSH foil **mesh**: change angle of attack with geometry rotation
- ▶ Describe two different solutions (OF module **foamRun**)
 - ▶ steady state – **SIMPLE** algorithm
 - ▶ transient – **PIMPLE** algorithm
- ▶ show mesh generation for different angles of attack (compare with fixed mesh),
- ▶ solution initialization with **potentialFoam** integrated in **foamRun**,
- ▶ compare solutions of the case for angle of attack 20° (turbulent and laminar solver).

Simple mixer case



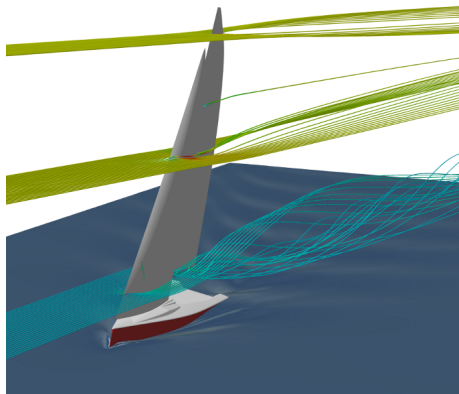
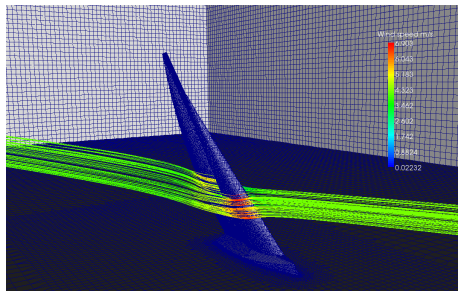
Problem: mixing dye with paint

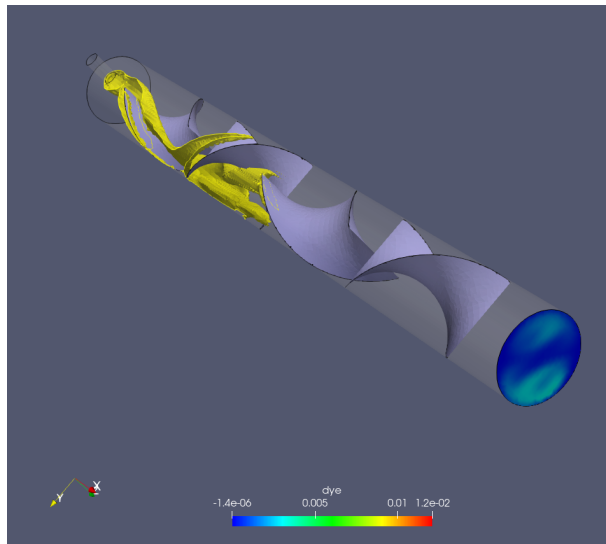
- ▶ steady state – SIMPLE algorithm – show run without the dye, only paint flow
- ▶ transient – PIMPLE algorithm – run complete case with dye inflow until 300s

Meshing procedure

- ▶ **blockMesh**: background mesh procedure
- ▶ **snappyHexMesh**: steps to generate computational mesh
 - ▶ **baffles** objects in OpenFOAM mesh (2D object in 3D space)
 - ▶ **surfaceFeatures** detect featured edges
 - ▶ set **refinement** levels
 - ▶ use **snappyHexMeshConfig** to generate automatically dictionary files
 - ▶ edit all generated dictionary files

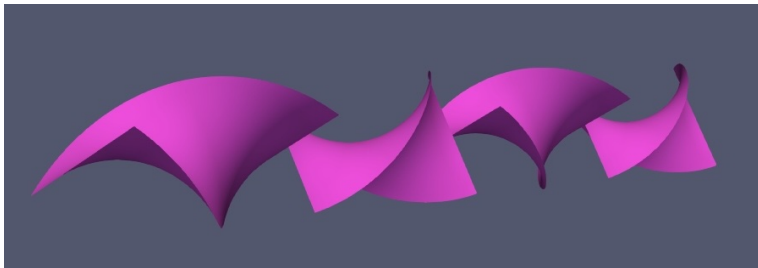
Typical is simulation of sail performance. It is a 2D object in 3D space





For 2D geometry embedded into 3D space in OpenFOAM snappyHexMesh use **baffle** element type.

In mixer baffles are static curved surfaces installed in pipe to mix dye with paint.





- ▶ `snappyHexMeshConfig -help`
- ▶ Set baffles from file `constant/geometry/baffles.obj`
`$> snappyHexMeshConfig -baffles '(baffles)'`

Surface geometry files are automatically recognised; look at the output

```
+ pipe
+ File: "pipe.obj"
+ Bounding box: (0 -0.05 -0.09) (1 0.05 0.05)
+ Closed surface
+ External boundary surface
+ Inlet regions: inletMain, inletSide
+ Outlet regions: outlet

+ baffles
+ File: "baffles.obj"
+ Bounding box: (0.1 -0.05 -0.05) (0.9 0.05 0.05)
+ Open surface
+ Baffle wall surface
```



snappyHexMeshConfig generates 3 system mesh files

- ▶ blockMeshDict
- ▶ surfaceFeaturesDict
- ▶ snappyHexMeshDict

blockMeshDict

geometry file `pipe.obj` forms an external boundary so,
it is possible to clear the boundary

```
$> snappyHexMeshConfig -baffles '(baffles)' -clearBoundary
```

Comment on blockMeshDict file



surfaceFeaturesDict file containing surface feature capturing information
generates additional geometry files with extension ***.eMesh**

```
$> snappyHexMeshConfig -baffles '(baffles)'  
      -clearBoundary -explicitFeatureSnap
```

- ▶ **implicitFeatureSnap**: automatically identifies surface features
 - ▶ may not pickup specific features, like ends, edges, ...
- ▶ **explicitFeatureSnap**: features defined in a file
 - ▶ user must specify a file
 - ▶ complete control over features identification and refinement



```
1 surfaces                                <-- list of surface geometry files
2 (
3     "pipe.obj"
4     "baffles.obj"
5 );
6
7 includedAngle    150;                  <-- feature identification parmeter
8
9 subsetFeatures
10 {
11     nonManifoldEdges yes;              <-- edges connected to more than 2 edges
12     openEdges       yes;              <-- edges connected to 1 face
13 }
14
15 trimFeatures
16
17 {
18     minElem        0;                  <-- minimum number of edges in feature
19     minLen          0;                  <-- minimum length of feature
20 }
21 writeObj          yes;                 <-- write additional obj files to visualise features
```


Generate all mesh dictionaries

```
1 snappyHexMeshConfig \  
2   -baffles '(baffles)' \  
3   -clearBoundary \  
4   -explicitFeatures \  
5   -refinementLevel 1 \.          <-- default level  
6   -surfaceLevels '((pipe 2))' \  <-- pipe refinement level  
7   -refinementRegions '((pipe 1))' \ <-- internal region  
8   -nCellsBetweenLevels 1        <-- cells between refinement levels
```

To create the mesh, must run

- ▶ blockMesh
- ▶ surfaceFeatures
- ▶ snappyHexMesh -overwrite (overwrite creates polyMesh)

Transport properties are set in dictionaries

- ▶ `constant/physicalProperties:`
transport material properties: viscosity, density, ...
- ▶ `constant/momentumTransport:`
fluid flow momentum transport properties: flow type, turbulence model, viscosity model, ...
- ▶ `constant/thermophysicalTransport:`
energy transport properties: specific conduction, enthalpy, ...



Paint can be described as **generalized newtonian fluid**, viscosity is not constant, but strain rate $\dot{\gamma}$ dependant

$$\nu = f(\dot{\gamma}), \quad \dot{\gamma} = \sqrt{2}|\mathbf{D}|, \quad \mathbf{D} = \text{sym}(\nabla \mathbf{v})$$

where: $\dot{\gamma}$ is strain rate, $\mathbf{D}$ is symmetric part of velocity gradient

Shear stress

$$\boldsymbol{\tau} = 2\rho\nu\mathbf{D}$$

Listing models

```
$> foamToC -table generalisedNewtonianViscosityModel
```

```
$> foamInfo BirdCarreau
```



```
1 Contents of table generalisedNewtonianViscosityModel:
2     BirdCarreau                                libmomentumTransportModels.so
3     Casson                                     libmomentumTransportModels.so
4     CrossPowerLaw                             libmomentumTransportModels.so
5     HerschelBulkley                           libmomentumTransportModels.so
6     Newtonian                                libmomentumTransportModels.so
7     powerLaw                                  libmomentumTransportModels.so
8     strainRateFunction                        libmomentumTransportModels.so
```

For **paint** use the **Bird-Carreau** strain rate $\dot{\gamma}$ dependant viscosity model

$$\nu = \nu_{\infty} + (\nu_0 - \nu_{\infty})[1 + (k \dot{\gamma})^a]^{(n-1)/a}$$

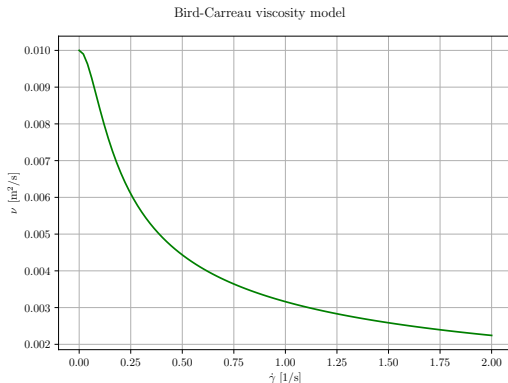
$$\nu_0 = 0.01 \text{ m}^2/\text{s}$$

$$\nu_{\infty} = 0.00001 \text{ m}^2/\text{s}$$

$$k = 10$$

$$n = 0.5$$

$$a = 2 \quad (\text{by default})$$





Choice of two transport models

- ▶ **laminar**: slow fluid flow, minimal vorticity
- ▶ **turbulent**: when not laminar

All transport models are set in the `constant/momentumTransport` file.



```
1 simulationType  laminar;  
2  
3 laminar  
4 {  
5     model          generalizedNewtonian;  
6  
7     viscosityModel  BirdCarreau;  
8     nuInf           1e-5;  
9     k               10;  
10    n               0.5  
11 }
```



```
1 simulationType RAS;  
2  
3 RAS  
4 {  
5     model          kOmegaSST;  
6     turbulence      on;  
7     printCoeffs     on;  
8  
9     viscosityModel  BirdCarreau;  
10    nuInf           1e-5;  
11    k               10;  
12    n               0.5  
13 }
```




Simulate flow of paint only. Test laminar and turbulent transport model.

Transport model

- ▶ **laminar**: iterate until 250 steps
- ▶ **turbulent**: iterate until 250 steps

Inlet velocity

- ▶ **main**: 1.0 m/s
- ▶ **side**: 0.5 m/s (dye inflow is zero!)

Comment on constant and system files!



Include dye flow at side inlet. Test laminar and turbulent transport model.

Transport model

- ▶ **laminar**: iterate until 2.0 s
- ▶ **turbulent**: iterate until 2.0 s

Scalar inlet – Dye source

- ▶ **main**: 0.0
- ▶ **side**: 1.0

Dye (scalar field) dimension is arbitrary. Normally dimension less.

Comment on constant and system files!



- ▶ for transient simulation **PIMPLE** solver is used
- ▶ for steady-state simulation **SIMPLE** solver is used

Courant number or **CFL** number

- ▶ **definition:**

$$\text{CFL} = \frac{U_x \Delta t}{\Delta x}$$

- ▶ U_x : velocity in cell
 - ▶ Δt : time step
 - ▶ Δx : cell size in flow direction
- ▶ **explicit** solver: $\text{CFL} < 1$
- ▶ **implicit** solver: CFL is more an accuracy consideration



Use some rough approximations in the estimation

- ▶ use `checkMesh` to find cell size Δx

$$\text{Min volume} = 1.7680364\text{e-}09 \quad \Delta x = \sqrt[3]{2e-9} \approx 1.3 \text{ mm}$$

- ▶ steady state max velocity $\approx 2.0 \text{ m/s}$

- ▶ estimated initial **time step**:

$$\Delta t = \text{CFL} \frac{\Delta x}{U_x} = 1.0 \frac{1.3 \text{ mm}}{2.0 \text{ m/s}} \approx 0.6 \text{ ms}$$



Try to play with `maxCo` (max CFL) in `system/controlDict`.

Warning: change in CFL is related to simulation accuracy.

Test dye travel distance for concentration 0.05!

Run simulation with different CFL and measure time of simulation

- ▶ use `maxCo = 1.0`
- ▶ use `maxCo = 2.0`
- ▶ use `maxCo = 4.0`

Use `foamLog` utility to extract simulation step data and plot using `GnuPlot`!

```
1 File: system/controlDict
2
3 adjustTimeStep yes; // time adaptive scheme ON (use CFL limit)
4 maxCo          1;   // max CFL number {1,2,4}
5
6 Follow:
7 - time of execution
8 - number of iterations in U and p
```

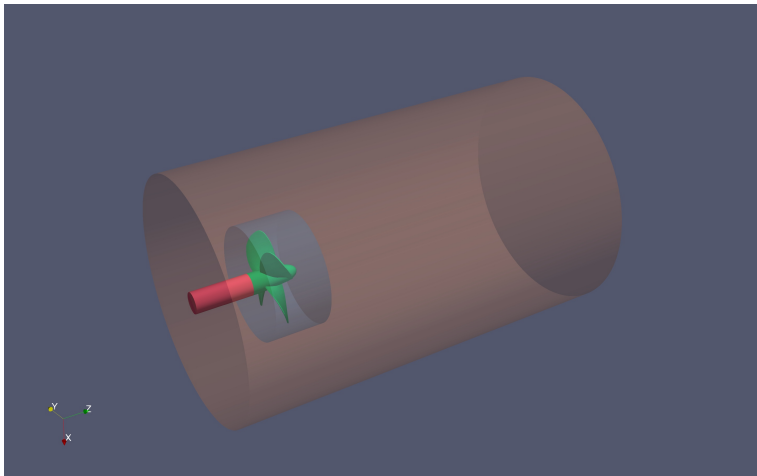
Ship resistance case



Case: Calculation of ship resistance force

- ▶ describe basic case philosophy
- ▶ describe meshing approach
 - ▶ **blockMesh**: anisotropic background mesh
 - ▶ **refineMesh**: different refinement mesh regions
 - ▶ **snappyHexMesh**: addition of boundary layer
 - ▶ **additional feature**: mesh refinement in bow and stern region (TODO)
- ▶ compute quasi steady state
 - ▶ steady state – **PIMPLE** algorithm with **localEuler** – pseudo transient
 - ▶ **setFields**: set air and water region
 - ▶ **incompressibleVoF**: solver module for 2 incompressible, isothermal immiscible fluids using a VOF (volume of fluid) phase-fraction based interface capturing approach

Propeller – Open water test case



case zones: external, rotating, shaft, propeller



MRF approach.

► **velocity**

velocity is composed of two terms: **relative** and **rotating**

$$\mathbf{v} = \mathbf{v}_{\text{rel}} + \boldsymbol{\omega} \times \mathbf{r}$$

$\mathbf{v}_{\text{rel}}$ - relative velocity (inlet flow)

$\boldsymbol{\omega}$ - angular velocity (propeller rotation)

$\mathbf{r}$ - radius vector in rotating zone

► **momentum** convection

use of a relative velocity for convective flux

$$\left(\mathbf{v} \cdot \nabla \right) \mathbf{v} = \left(\mathbf{v}_{\text{rel}} \cdot \nabla \right) \mathbf{v} + \boldsymbol{\omega} \times \mathbf{v}$$



Case: Calculation of ship propeller thrust/moment.

- ▶ describe basic case philosophy
- ▶ describe meshing approach
 - ▶ **blockMesh**: radially symmetric background mesh
 - ▶ **rotatingZone**: define rotating mesh regions
 - ▶ **snappyHexMesh**: rotating zone
 - ▶ **additional feature**: mesh refinement only in blade region and boundary layer (TODO)
- ▶ compute steady state
 - ▶ steady state – **PIMPLE** algorithm with **MRF** source
 - ▶ **movingWallVelocity**: set as velocity BC at rotating surface (propeller) to obtain rotation
 - ▶ **incompressibleVoF**: solver module for 2 incompressible, isothermal immiscible fluids using a VOF (volume of fluid) phase-fraction based interface capturing approach



Thank you for attention!



EuroHPC
Joint Undertaking



REPUBLIC OF SLOVENIA
MINISTRY OF HIGHER EDUCATION,
SCIENCE AND INNOVATION

This project has received funding from the European High-Performance Computing Joint Undertaking (JU) under grant agreement No 101101903. The JU receives support from the Digital Europe Programme and Germany, Bulgaria, Austria, Croatia, Cyprus, Czech Republic, Denmark, Estonia, Finland, Greece, Hungary, Ireland, Italy, Lithuania, Latvia, Poland, Portugal, Romania, Slovenia, Spain, Sweden, France, Netherlands, Belgium, Luxembourg, Slovakia, Norway, Türkiye, Republic of North Macedonia, Iceland, Montenegro, Serbia.